Neural Network Ensembles and Stacking for Network Traffic Classification

Student number: 27233162

***Abstract*—Ensemble learning was investigated on 257 673 network connection records from the UNSW-NB15 dataset to test whether dissimilar classifiers improved traffic classification. A weight-sharing neural network ensemble called TabM was evaluated alone and within a stack. The stack also contained the re-tuned nearest neighbour and decision tree models of an earlier study. The comparison reused the earlier grouped five-fold evaluation, with nine questions guiding tuning and per-class analysis. The three-learner stack had the highest mean macro F-score of 0.603 ± 0.010, against 0.575 ± 0.017 for the earlier tree. The stack scored higher than the earlier tree on every fold. The difference was not significant under the corrected resampled t-test, with $p = 0.076$. TabM alone had a macro F-score of 0.525 but improved the stack on every fold. The improvement was consistent with complementary errors of TabM and the re-tuned tree. The re-tuned tree recovered more minority attacks, whereas TabM classified frequent classes more accurately. Learned stacking combined the two precision–recall trade-offs. Equal-weight averaging remained competitive. No ensemble improved every attack class. Class weights, distinct minibatches and dropout lowered the validation macro F-score of TabM during tuning. The findings applied to one dataset and a protocol revised after the first test results were known.**

***Index Terms*—ensemble learning, stacked generalisation, neural networks, network traffic classification**

I. Introduction

The strongest individual classifier need not supply every useful prediction for an ensemble. An earlier study of network traffic classification found that a decision tree outperformed k-nearest neighbours (k-NN) in macro F-score. However, k-NN performed better on the denial of service (DoS) and Analysis attack classes. These class-specific differences motivated a comparison of models with different inductive biases, the assumptions about the input–output relationship that guide learning [12].

The present study compared TabM and stacked generalisation with the earlier models. TabM combines multilayer perceptrons (MLPs) through shared weight matrices [6]. Stacked generalisation, or stacking, trains a meta-learner on predictions from several base learners [20]. The study deliberately chose members dissimilar to the earlier models. A random forest or boosted trees would extend the partitioning rules of the earlier tree to address variance or bias, but trees grown on the same data remain correlated, and correlation limits the gain from averaging [8]. An ensemble outperforms the individual members when those members are accurate and make different errors [4]. An MLP learns a continuous mapping from all features jointly, an inductive bias distinct from the local neighbourhoods of k-NN and the feature-specific splits of a tree [7]. The chosen comparison asked whether a neural learner supplied useful information beyond the two earlier models.

The research question was whether either ensemble improved the macro F-score on the cross-validation folds of the earlier study. The study also asked what the tuning revealed about the differences. Nine questions addressed ensemble size, weighting, member diversity, capacity, the learned combination and class-specific performance. Reproducing and re-tuning the earlier models provided the original baselines and stronger individual learners for the ensemble comparison.

Stacking had the highest mean macro F-score, but the comparison did not establish a significant gain over the earlier tree. A weaker TabM improved the stack on every fold, showing that standalone ranking did not describe the usefulness of a base learner. Section II gives the model background. Sections III and IV describe the implementation and empirical process. Section V interprets the results, and Section VI concludes the report.

II. Background

This section describes the classification task and the two models of the earlier study. The section then covers the principles of ensemble learning, the two ensembles and the tests for comparing models over cross-validation folds.

A. Network Traffic Classification

Network traffic classification assigns each connection record to the normal class or to an attack category. The input consists of recorded flow statistics, such as byte counts, packet rates, duration and protocol. The task therefore concerns classification from summarised connection attributes rather than from the original packet contents [13]. Attacks that differ mainly in packet contents can therefore produce identical attribute values, so no classifier that uses these inputs can separate every observation.

B. Nearest Neighbour Classification

The k-NN classifier predicts the label of a query observation from the labels of the most similar training observations [3]. The method stores the n training pairs $(\mathbf{x}_i, y_i)$ and fits no parameters. The method rests on the assumption that observations close together in the feature space tend to belong to the same class. A distance function makes closeness measurable. The function reduces all feature differences between two observations to one non-negative number, and identical observations have distance zero. The choice of distance function

therefore determines the training observations that decide each prediction.

For two vectors $\mathbf{x}$ and $\mathbf{x}'$ with m features, the Minkowski distance of order q is

$$d(\mathbf{x}, \mathbf{x}') = \Big(\sum_{j=1}^{m} |x_j - x'_j|^q \Big)^{1/q}, \tag{1}$$

where x_j is feature j of $\mathbf{x}$. The order $q = 1$ gives the Manhattan distance, the sum of the absolute differences along the feature axes. The order $q = 2$ gives the Euclidean distance, the straight-line length between two points. The Manhattan distance counts every unit of difference equally, whatever the feature. The Euclidean distance squares the differences, so one large difference counts for more than several small differences with the same sum. A larger order gives still more weight to the largest single difference.

Every feature enters the sum on the numerical scale of the feature, so a feature with a range of millions dominates a feature with a range of one. Numeric features are therefore rescaled to comparable ranges before distances are computed [8]. Every feature also counts in the distance, whether or not the feature carries class information. Irrelevant features add variation to every distance and so blur the contrast between near and distant observations. In many dimensions, a neighbourhood must also span a large part of the range of every feature to contain a fixed share of the training observations. The nearest observations are then no longer local [8].

Nominal features have no numerical order. One-hot encoding replaces a nominal feature with one binary indicator for each level. Two observations with different levels then differ in two indicators, and the Manhattan distance between the observations grows by two. Every pair of distinct levels therefore contributes the same amount to the distance.

To classify a query observation $\mathbf{x}$, the method finds the index set $N_k(\mathbf{x})$ of the k training observations nearest to $\mathbf{x}$. Each neighbour i casts a vote with weight w_i for the class y_i. The estimated probability of class l is the weighted share of the votes,

$$\hat{\pi}_l(\mathbf{x}) = \frac{\sum_{i \in N_k(\mathbf{x})} w_i \, \delta(y_i, l)}{\sum_{i \in N_k(\mathbf{x})} w_i}, \tag{2}$$

where $\delta(y_i, l)$ is one when $y_i = l$ and zero otherwise. The predicted class $\hat{y}$ is the class with the largest estimated probability. Uniform weighting sets $w_i = 1$. Distance weighting sets $w_i = 1/d(\mathbf{x}, \mathbf{x}_i)$, so closer neighbours count more. The weight is undefined at zero distance, so scikit-learn lets only zero-distance neighbours vote when any neighbour has zero distance. With $k = 3$ and uniform weights, every probability is a multiple of one third, so the estimates are coarse. Algorithm 1 summarises the classification of one query observation.

The number of neighbours k controls a trade-off between bias and variance [8]. With $k = 1$, the nearest training observation alone decides the prediction. Every query observation whose nearest neighbour is mislabelled then receives the wrong class. With a larger k, a mislabelled neighbour supplies one of k votes and changes the prediction only when the other neighbours split their votes. A larger k therefore averages out label noise and isolated atypical observations, and the boundaries between class regions become smoother.

Algorithm 1 Nearest neighbour classification of one query observation

1: Store the n training observations and labels.
2: **for each** training observation i **do**
3: Compute the distance $d(\mathbf{x}, \mathbf{x}_i)$ to the query observation $\mathbf{x}$.
4: **end for**
5: Form $N_k(\mathbf{x})$ from the k observations with the smallest distances.
6: Give each neighbour i the vote weight w_i.
7: Estimate $\hat{\pi}_l(\mathbf{x})$ for each class l from the weighted votes.
8: Predict the class with the largest estimated probability.

The cost of a larger k is bias, because a large neighbourhood reaches into the regions of other classes. A neighbourhood larger than the cluster of a rare class necessarily contains observations of other classes, so frequent classes nearby compete with the rare class in every vote. Distance weighting reduces the bias, because nearer neighbours count more. Under distance weighting, a mislabelled observation very close to the query also keeps a large weight. The neighbourhood is ambiguous when several training observations lie at exactly the distance of the k-th neighbour. Any subset of the tied observations completes a valid neighbourhood, and different implementations choose different subsets.

C. Decision Tree Classification

A classification tree partitions the feature space with a sequence of binary tests [1]. Each internal node tests one feature against a threshold, $x_j \le \theta$, and sends an observation to the left or the right child. Each leaf stores the class proportions of the training observations that reach the leaf. A query observation follows the tests from the root to a leaf. The tree predicts the most frequent class of the leaf, and the leaf proportions serve as class probability estimates.

Growing the tree selects, at each node t, the test that makes the child nodes as pure as possible. Impurity measures how mixed the classes are among the training observations S_t at node t. Impurity is zero when all observations at the node share one class, and maximal when all K classes are equally frequent. Let $\pi_l(t)$ be the proportion of class l among the observations in S_t. The Gini impurity

$$G(t) = 1 - \sum_{l=1}^{K} \pi_l(t)^2 \tag{3}$$

is the probability that two observations drawn at random with replacement from S_t belong to different classes. The entropy

$$H(t) = -\sum_{l=1}^{K} \pi_l(t) \log_2 \pi_l(t) \tag{4}$$

is the average information content, in bits, of the class of an observation drawn from S_t [8]. A test sends the observations S_L to the left child t_L and the observations S_R to the right child t_R. For unweighted observations and impurity Q equal to G or H, the impurity decrease is

$$\Delta Q = Q(t) - \frac{|S_L|}{|S_t|} Q(t_L) - \frac{|S_R|}{|S_t|} Q(t_R). \tag{5}$$

The algorithm evaluates every candidate feature and threshold, applies the test with the largest decrease and repeats the search in each child.

Impurity matters because the impurity decrease chooses the test at every node, and each chosen test constrains all descendant nodes. A pure leaf predicts one class with probability one, whereas an impure leaf gives graded probabilities. The misclassification rate of a node would be a simpler criterion, but the rate fails to separate many candidate tests. Hastie *et al.* illustrated the problem with 400 observations of each of two classes [8]. A split into nodes with class counts $(300, 100)$ and $(100, 300)$ misclassified 200 observations, as did a split into nodes with counts $(200, 400)$ and $(200, 0)$. Only the second split created a pure node, and both the Gini impurity and the entropy were lower for the second split. Impurity therefore rewards progress towards pure nodes that the misclassification rate ignores. No test separates observations with identical feature values, so a node that holds identical observations with different labels keeps a positive impurity at any depth.

Unrestricted growth fits increasingly specific patterns in the training observations, including label noise. Maximum depth and minimum node sizes limit growth. Cost-complexity pruning balances fit against the number of leaves [1]. Pruning minimises $e(T) + \alpha|T|$, where $e(T) = \sum_t (n_t/n)\, Q(t)$ sums the leaf impurities weighted by the share n_t/n of training weight in leaf t, $|T|$ counts the leaves and $\alpha \geq 0$ penalises each leaf. The classical formulation measures $e(T)$ with the misclassification rate, whereas scikit-learn uses impurity. Algorithm 2 summarises the growth and pruning of a tree.

Class weights ω_l multiply the contribution of each training observation of class l. Balanced weights $\omega_l = n/(Kn_l)$, where n_l counts training observations of class l, give every class the same total weight n/K. The weights affect class proportions, impurity, split selection and pruning. Child-size ratios in the impurity decrease then use total weights rather than observation counts. Weighting therefore changes both the partition and the leaf predictions. A tree depends only on the ordering of the values of each feature, not on the numerical scale, but small changes to the data can change the selected splits [8].

Algorithm 2 Growth and pruning of a classification tree

```
Place all training observations in the root node.
for each node t, in the order of creation do
    if t is pure, has the maximum depth, holds fewer observations than the minimum split size or allows no valid test then
        Make t a leaf that stores the class proportions π_l(t).
    else
        Compute ΔQ for every candidate feature j and threshold θ.
        Apply the test x_j ≤ θ with the largest ΔQ among the tests that leave at least the minimum leaf size in each child.
        Add the two child nodes.
    end if
end for
Prune the tree to the subtree T that minimises e(T)+α|T|.
```

D. Ensemble Learning

An ensemble combines the predictions of several models, called members or base learners. A combination of members reduces prediction variance when member errors differ. A combination also represents decision boundaries beyond the capacity of one member. Dietterich attributed ensemble benefits to statistical, computational and representational causes [4]. Members must make useful predictions and differ in errors for a combination to improve classification [4]. Kuncheva and Whitaker nevertheless found no consistent relationship between diversity measures and ensemble accuracy [10].

Kuncheva and Whitaker compared diversity measures defined on whether each member of a pair classifies an observation correctly [10]. Their disagreement is the share of observations that exactly one of the two members classifies correctly. The double fault is the share of observations that both members misclassify. The error correlation is the correlation between the binary error indicators of two members. This study reported label disagreement instead, the share of observations where two members predict different labels, so observations that both members misclassify with different labels also counted.

Homogeneous ensembles use one model family. Bagging trains each member on a bootstrap sample and averages the members, mainly reducing prediction variance. Random forests add a random choice of candidate features at each split. In AdaBoost, boosting trains members in sequence and raises the weights of misclassified observations [8]. Heterogeneous ensembles combine different algorithms, so the members differ in inductive bias even when trained on the same observations [20]. The simplest combinations are the majority vote of predicted labels and the soft vote, the average of class probabilities.

E. Stacked Generalisation

Wolpert introduced stacked generalisation [20]. In stacking, a meta-learner learns to combine the predictions of base learners. The meta-features, the inputs of the meta-learner, are the predictions of the base learners for each training observation. The meta-features must come from base learners that did not see the observation. A fully grown tree fits the training observations of the tree almost perfectly, so in-sample meta-features would teach the meta-learner to trust the

most overfitted base learner [8]. Stacking therefore obtains out-of-fold meta-features by an internal cross-validation, as Algorithm 3 shows. Each base learner is then refitted on all training observations to produce the meta-features of new observations.

Ting and Witten found that stacking performed best when the meta-features were class probabilities rather than predicted labels [17]. Probabilities carry the confidence of each base learner, whereas labels discard the confidence. Any learning method can serve as the meta-learner, including a multinomial logistic regression [8]. Let $\mathbf{z}$ be the meta-feature vector of an observation. The model estimates the probability of class l as

$$\hat{\pi}_l(\mathbf{z}) = \frac{\exp(\boldsymbol{\beta}_l^{\mathsf{T}}\mathbf{z} + \beta_{l,0})}{\sum_{l'=1}^{K} \exp(\boldsymbol{\beta}_{l'}^{\mathsf{T}}\mathbf{z} + \beta_{l',0})}, \tag{6}$$

with coefficient vector $\boldsymbol{\beta}_l$ and intercept $\beta_{l,0}$ for class l [8]. Fitting minimises the penalised negative log-likelihood

$$\frac{1}{2}\sum_{l=1}^{K} \|\boldsymbol{\beta}_l\|^2 - C\sum_{i=1}^{n} \ln \hat{\pi}_{y_i}(\mathbf{z}_i), \tag{7}$$

where a smaller inverse regularisation strength C shrinks the coefficients more. An own-class coefficient links the probability of class l from a base learner to the linear score $\boldsymbol{\beta}_l^{\mathsf{T}}\mathbf{z} + \beta_{l,0}$ of class l. The final decision depends on all coefficients and all class scores. Coefficient magnitude alone therefore does not measure the importance of a learner.

F. Multilayer Perceptrons and Weight-Sharing Ensembles

An MLP maps an input vector through a sequence of layers [5]. Each hidden layer computes $\mathbf{a}' = \max(\mathbf{0}, \mathbf{W}\mathbf{a} + \mathbf{b})$ from the layer input $\mathbf{a}$, with a weight matrix $\mathbf{W}$, a bias vector $\mathbf{b}$ and the elementwise rectified linear unit (ReLU). A final linear layer produces one score for each class, and the softmax function turns the scores into class probabilities. Training minimises the cross-entropy, the mean of $-\ln \hat{\pi}_y(\mathbf{x})$ over the training observations, with minibatch gradient steps. The adaptive moment estimation (Adam) optimiser adapts the step size of every weight. Dropout sets randomly chosen hidden units to zero during training and so discourages co-adapted units. Early stopping keeps the weights from the epoch with the best validation score [5]. Adam with decoupled weight decay (AdamW) shrinks the weights directly at each step instead of adding the decay to the gradient that Adam rescales [11].

On medium-sized tabular datasets of about 10 000 training observations, tree ensembles outperformed neural networks, including MLPs, in a benchmark by Grinsztajn *et al.* [7]. The authors attributed the gap to the preference of MLPs for smooth functions and to the sensitivity of MLPs to uninformative features.

An ensemble of independently trained networks requires a separate set of weights and a separate fit for every member. BatchEnsemble reduces this cost by sharing one weight matrix $\mathbf{W}$ in each layer among M members [18]. Member h owns two scaling vectors, $\mathbf{r}_h$ and $\mathbf{s}_h$, and a bias vector $\mathbf{b}_h$. For the input $\mathbf{a}_h$ of member h, the layer computes the output

$$\mathbf{o}_h = \mathbf{s}_h \odot \big(\mathbf{W}(\mathbf{r}_h \odot \mathbf{a}_h)\big) + \mathbf{b}_h, \tag{8}$$

where $\odot$ is the elementwise product. The output equals the output of an ordinary linear layer with the member weight matrix $\mathbf{W}_h = \mathbf{W} \odot (\mathbf{s}_h \mathbf{r}_h^{\mathsf{T}})$. Each member therefore uses the shared matrix rescaled by a member-specific matrix of rank one. All members run as one batched matrix product, and each member adds only two vectors and a bias to the shared weights.

TabM uses BatchEnsemble layers in the hidden network and a separate output layer for each member [6]. All members train simultaneously. Training batches are either shared or sampled through separate shuffles for the members. Let $\hat{\pi}_{h,l}(\mathbf{x})$ be the probability that member h assigns to class l. The loss for an observation with label y is the mean of the member cross-entropies,

$$\mathcal{L}(\mathbf{x}, y) = -\frac{1}{M}\sum_{h=1}^{M} \ln \hat{\pi}_{h,y}(\mathbf{x}). \tag{9}$$

The prediction averages the member probabilities, $\hat{\pi}_l(\mathbf{x}) = \frac{1}{M}\sum_{h=1}^{M} \hat{\pi}_{h,l}(\mathbf{x})$. The logarithm is concave, so by the Jensen inequality the cross-entropy $-\ln \hat{\pi}_y(\mathbf{x})$ of the averaged prediction does not exceed $\mathcal{L}(\mathbf{x}, y)$. A loss on the averaged prediction tolerates weak members when other members compensate. The mean member loss instead penalises poor predictions by each member directly. Algorithm 4 summarises training with early stopping. Gorishniy *et al.* reported that TabM performed better on average than earlier tabular deep learning models across the benchmark datasets [6].

G. Statistical Comparison over Folds

Two models evaluated on the same V cross-validation folds give one paired score difference D_v for each fold v. The Wilcoxon signed-rank test ranks the absolute differences and compares the rank sums of the positive and the negative differences. The test does not require normally distributed

Algorithm 3 Stacked generalisation with out-of-fold meta-features

1: Split the training observations into internal folds.
2: **for each** internal fold **do**
3: **for each** base learner **do**
4: Fit the base learner on the other internal folds.
5: Predict class probabilities for the held-out fold.
6: **end for**
7: **end for**
8: Join the probabilities of all base learners into one meta-feature vector for each training observation.
9: Fit the meta-learner on the meta-features and the true labels.
10: Refit every base learner on all training observations.
11: Predict a new observation with the meta-learner from the meta-features that the refitted base learners produce.

Algorithm 4 Training of TabM with early stopping

```
Initialise the shared weights and the member vectors.
for each epoch do
    for each minibatch of training observations do
        Pass the minibatch through all members at once.
        Compute the mean of the member cross-entropies.
        Update all weights with one optimiser step.
    end for
    Score the averaged member probabilities on the validation observations.
    Keep the weights if the score is the best so far.
    Stop if the score has not improved for a set number of epochs.
end for
Restore the weights with the best validation score.
```

differences, but assumes independent differences that are symmetric about their median under the null hypothesis [19]. The exact two-sided test cannot give a p-value below $2/2^V$, and the bound is 0.0625 for five folds.

The training sets of different folds overlap, so the fold scores are correlated. The ordinary paired t-test then underestimates the variance of the mean difference. Nadeau and Bengio proposed the corrected resampled t-test [14]. With the mean $\bar{D}$ and the sample variance $\hat{\sigma}^2$ of the V differences, the statistic is

$$\tau = \frac{\bar{D}}{\sqrt{\left(\frac{1}{V} + \frac{1}{V-1}\right)\hat{\sigma}^2}}. \tag{10}$$

The term $1/(V-1)$ is the ratio of test observations to training observations in V-fold cross-validation. The statistic τ approximately follows a Student t distribution with $V-1$ degrees of freedom.

Testing several comparisons raises the probability that at least one comparison appears significant by chance. The Holm procedure sorts the p-values of a family of comparisons in increasing order. The smallest value is multiplied by the number of comparisons, and each later value by one less than the multiplier of the preceding value. Any adjusted value below the preceding adjusted value is raised to the preceding value, and no adjusted value exceeds one. Holm showed that rejecting every hypothesis with an adjusted value below the significance level keeps the probability of any false rejection at or below that level [9].

In summary, k-NN predicts from the labels of the nearest training observations under a chosen distance. The number of neighbours k balances the averaging of label noise against bias towards frequent classes. A decision tree predicts from the class proportions of the leaf that an observation reaches, and the impurity decrease selects every test. Ensembles gain accuracy from members with different errors. Stacking learns the combination from out-of-fold predictions, and TabM limits the parameter count by sharing weights among the MLPs. Paired tests over folds compare the models, with a corrected variance for overlapping training sets and the Holm adjustment for multiple comparisons.

III. Implementation

This section describes the software, the structure of the code and the shared preprocessing. Models were scored on five outer cross-validation folds. Tuning used three inner folds within the training partition of each outer fold, called the outer-training partition. The section then gives the implementation of each model, with the checks used to verify each part.

A. Software and Code Structure

The experiments ran in one Python 3.12.3 notebook on an NVIDIA GeForce RTX 5070 Ti graphics processing unit (GPU). Scikit-learn 1.9.0 [16] supplied the preprocessing, both trees, the baseline k-NN, logistic regression and the cross-validation splitters. PyTorch 2.13.0 [15] ran TabM and the revised neighbour search on the GPU. The tabm 0.0.3 package supplied TabM.[1] NumPy 2.4.6, pandas 3.0.3 and SciPy 1.18.1 handled data and statistics. The default random seed was 21, matching the earlier study.

The notebook followed the order of the study, from fold reconstruction and model reproduction to re-tuning, TabM training, stacking and analysis. One shared function produced the out-of-fold and outer-test probabilities of every base learner. The function received a model-specific routine that fitted a model on given observations and returned class probabilities for other observations. All base learners therefore passed through the same fold loop, and one leakage check of the loop covered every learner. The notebook stored the probability tables, tuning results and timings of every outer fold on disk. The stacks, tests and figures reused the stored tables instead of refitting the base learners. Standalone scripts repeated the neighbour diagnostics and produced the figures in this report. The code and recorded environment are available in a public repository.[2]

B. Encoding and Scaling

The k-NN and tree pipelines reused the preprocessing code of the earlier study. Every transformation was fitted on the training partition of each fit and applied unchanged to the observations being predicted, so no statistic of a test observation entered training. One-hot encoding created a binary indicator for every level of `proto`, `state` and `service`. A level absent from the training observations received zeros in every indicator of the feature. Optional pooling merged the levels with fewer than 50 training observations into one indicator for each feature, and unseen levels then joined the pooled indicator.

Only `service` contained missing values, marked by a question mark in more than half of the observations. A constant-value imputer replaced every missing value with an additional level, `__missing__`, before encoding. One-hot encoding therefore added a missing-service indicator, with a

[1] https://github.com/yandex-research/tabm

[2] https://github.com/matt-roux/ml441-assignment4

one for every observation without a recorded service. A single tree test on that indicator separated all such observations. For k-NN, a missing and a recorded service differed by two in Manhattan distance, as for any other pair of levels with separate indicators. The share of observations without a recorded service ranged from 1.6% for Generic to 100% for Shellcode, so the missing level carried class information. Imputing the most frequent service would have removed that information, and deleting incomplete observations would have discarded more than half of the data.

The numeric features were strongly right-skewed, with a median skewness of 8.0. For the source byte count `sbytes`, the standard deviation was 139 times the interquartile range, the distance between the 25th and 75th percentiles. Standardisation subtracts the training mean of a feature and divides by the training standard deviation. Standardisation would therefore compress the central half of the `sbytes` values into less than 0.01 units. Robust scaling instead subtracts the training median and divides by the training interquartile range. The central half of every feature then spans one unit, whatever the size of the extreme values. Extreme values keep large scaled values, so robust scaling does not clip outliers. The k-NN search selected between the two methods.

Six numeric features had zero interquartile range in every training partition, because more than three quarters of the values of each feature were zero. For these features, the scaler kept a scale of one and only subtracted the median. The response size `response_body_len` therefore kept byte counts of up to 6.6 million. For 89% of sampled pairs of observations with a non-zero response body, `response_body_len` supplied more than half of the Manhattan distance over the numeric features. Optional correlation pruning removed a numeric feature with an absolute Pearson correlation of at least 0.995 with a feature retained earlier in the column order. Optional univariate selection kept the 30 encoded features with the largest analysis-of-variance F statistics. The tree pipeline used the same encoding without scaling, because a threshold test on a rescaled feature splits the observations exactly as the matching test on the original feature.

C. Reproduction of the Earlier Models

The baseline models used the earlier fold-specific settings and the same outer-training observations. The tree reproduced every earlier test prediction. The k-NN predictions matched on four folds. On fold four, 20 of the 51 535 predictions differed. The macro F-score changed by 6.8×10^{-5}. The differences involved tied DoS and Exploits votes with alternative equidistant neighbours. The revised models used the same pipelines with settings selected on all outer-training observations.

D. Accelerated Neighbour Search

Full-scale tuning of k-NN required 768 fits for each outer fold, from 256 settings on three inner folds. One scikit-learn fit and prediction on the central processing unit (CPU) took 169 s, so the search would have taken more than a day for each outer fold. The revised search ran on the GPU with PyTorch. The implementation computed the distances between blocks of 256 query observations and all training observations in single precision. For each query observation, the implementation kept the 21 nearest training observations, ordered by distance and then by training index. Among observations tied at the k-th distance, the observations with the lowest training indices therefore entered the neighbourhood. Under this ordering, the k nearest observations were the first k of the 21 nearest observations. One search for each preprocessing setting, Minkowski order and inner fold therefore served all four values of k and both vote weightings. The 768 fits of each outer fold required only 96 neighbour searches.

The vote followed the rules of scikit-learn, including the exclusive vote of zero-distance neighbours under distance weighting. With the earlier fold-one setting, the implementation agreed with scikit-learn on 99.833% of labels.

Table I gives separate diagnostic checks with the earlier settings on all five folds. The cuML 26.8.0 library[3] ran faster but changed many labels, and every changed label involved a tie at the k-th distance. A second PyTorch diagnostic accumulated Manhattan distances in double precision before rounding to single precision and changed almost no labels. That diagnostic was not used for tuning or prediction. The table shows that the predicted labels depended on tie handling and arithmetic precision. The table does not include the single-precision search used in the experiments.

E. Weight-Sharing Neural Ensemble

TabM used model-specific preprocessing fitted on each training partition. A quantile transformation mapped each numeric feature to standard normal scores, as in the TabM experiments, where a slightly modified version of the transformation was used [6]. The scikit-learn defaults estimated 1 000 quantiles from a random subsample of 10 000 training observations. Rank-based transformation prevents heavy-tailed magnitudes from dominating the neural inputs. Nominal values received integer codes, and TabM applied one-hot encoding to the codes internally. As in the other pipelines, the missing service level received a separate code. Code zero marked levels unseen in training.

Each hidden block contained a BatchEnsemble linear layer, a ReLU and dropout. Width counted hidden units and depth counted blocks. Each member had a separate 10-output layer. The first input scaling vectors started with random signs, and the remaining multiplicative vectors started at one [6].

Training used AdamW with weight decay 3×10^{-4} and batches of 512 observations. Shared batches supplied identical observations to every member. Distinct batches used a separate shuffle for each member, with every member still seeing every training observation each epoch. When selected, balanced class weights scaled the cross-entropy contribution of each observation. Tuning kept the epoch with the best macro F-score on the inner validation fold. Training stopped after eight

[3]https://github.com/rapidsai/cuml

Table I
STANDALONE NEAREST NEIGHBOUR CHECKS ON THE FIVE OUTER FOLDS

Implementation	Seconds	Changed labels (%)	Largest change in macro F-score
Scikit-learn, CPU	169	reference	reference
cuML, GPU	30–35	6.0–9.7	0.0040
Double-precision, GPU	62	0.002–0.019	0.00003

epochs without improvement or after 80 epochs. Final fits used the rounded mean best epoch of the selected candidate without a hold-out.

Each final prediction averaged five independently trained TabMs with seeds 21 to 25. Ensemble size M referred to members within each TabM, so a final prediction combined $5M$ member outputs. A parameter count compared one TabM of width 512 and depth three with MLPs of the same size. With $M = 32$, the TabM had 927 776 parameters, 46% more than one MLP. Thirty-two independent MLPs would have had 20.3 million parameters.

With copied weights, the package layer used in training matched a hand-written implementation of the BatchEnsemble equation under an absolute tolerance of 10^{-6} and a relative tolerance of 10^{-5}. The loss equalled the mean member cross-entropy and equalled $\ln 10$ for zero scores. A four-member TabM fitted 64 non-conflicting observations to 100% training accuracy. These checks verified the layer, loss and ability to fit a small sample.

F. Stacking and Soft Voting

For each outer fold, a shared function generated out-of-fold base probabilities on three grouped inner folds. Each inner fit excluded the groups being predicted. Refitting on all outer-training observations produced the outer-test probabilities. On outer fold one, a leakage check substituted a memoriser for the base learner. The memoriser returned uniform probabilities on every held-out observation and confident predictions on fitted observations. The check verified direct group exclusion, but did not test independence of upstream parameter selection.

The meta-features were the base probabilities, the log-probabilities $\ln(\hat{\pi}_l + 10^{-4})$ or one-hot predicted labels. Standardisation preceded multinomial logistic regression with the limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) solver and a limit of 2 000 iterations. L-BFGS is a quasi-Newton method that approximates the curvature of the loss from the gradients of recent iterations, without storing a full curvature matrix [5]. Standardisation put all representations on one scale, so one regularisation grid served all three.

Stack S1 combined the revised k-NN and tree. Stack S2 added TabM. Soft vote V2 averaged the three base probability vectors with equal weights. Every model predicted the class with the largest probability and broke ties alphabetically.

The implementation preserved the earlier data transformations and outer folds. The direct checks supported the ensemble computations, whereas the neighbour diagnostics identified a small implementation difference that remained in the reported comparison.

IV. Empirical Process

This section introduces the supplied data and defines the partitioning, measures and tuning searches. The section then describes the revision after the first run and the statistical comparisons.

A. Dataset

UNSW-NB15 records normal connections and nine attack categories. Moustafa and Slay generated normal and synthetic attack traffic with IXIA PerfectStorm in the Cyber Range Lab at the University of New South Wales (UNSW) Canberra [13].[4] The traffic was captured with tcpdump, and Argus, Bro-IDS and additional feature-extraction algorithms produced connection-level records.

The supplied file contained 257 673 observations, one for each connection record. Removing the identifier left 39 numeric and three nominal predictors. Table II gives the size of each class, the share of observations without a recorded service and the share of observations in conflicting groups. Normal held more than 500 times as many observations as Worms, so class imbalance affected model selection. A conflicting group is a set of observations with identical predictor vectors and different labels. A deterministic classifier assigns one label to all observations of a conflicting group and so misclassifies every observation with another label. Conflicting groups held most of the Backdoor, DoS and Analysis observations.

B. Cross-Validation Protocol

Nested cross-validation separated parameter selection from outer-fold scoring [2]. Five grouped and stratified outer folds with seed 21 placed every observation in the same test fold as the earlier evaluation. Each group held all observations with one predictor vector, so no copy of a test observation entered training. Each test fold held 51 534 or 51 535 observations, including 34 or 35 Worms observations. Within each outer-training partition, three grouped inner folds served tuning and out-of-fold prediction.

The outer-test labels did not enter the fitting or parameter searches of the final run.

C. Performance Measures

For class l, precision P_l measures the fraction of observations predicted as class l that belong to the class. Recall R_l measures the fraction of the observations of class l that the model recovers. The class F-score $F_l = 2P_lR_l/(P_l + R_l)$ combines precision and recall, with zero assigned when no positive prediction is correct. The primary measure was the macro F-score

$$\bar{F} = \frac{1}{K}\sum_{l=1}^{K} F_l, \tag{11}$$

[4]https://research.unsw.edu.au/projects/unsw-nb15-dataset

Table II
OBSERVATIONS, MISSING SERVICE VALUES AND LABEL CONFLICTS OF EACH CLASS

Class	Observations	Missing service (%)	In conflicting groups (%)
Normal	93 000	68.7	1.0
Reconnaissance	13 987	84.2	16.1
Backdoor	2 329	95.3	84.9
DoS	16 353	84.7	77.4
Exploits	44 525	51.8	37.1
Analysis	2 677	78.9	77.0
Fuzzers	24 246	90.7	14.9
Worms	174	14.9	8.6
Shellcode	1 511	100.0	0.9
Generic	58 871	1.6	0.6
All classes	257 673	54.8	15.7

with $K = 10$. Balanced accuracy is the mean class recall. Accuracy is the fraction of correctly classified observations.

Macro F-score was the primary measure for three reasons. The study treated every attack category as equally important, whatever the frequency of the category. Macro averaging gives every class the same weight. The F-score of a class also falls with both missed observations and false predictions of the class, so a model cannot raise the score through recall alone. Finally, the earlier study selected and compared models by macro F-score, so the measure kept the baselines comparable.

Accuracy instead weights each class by frequency. Normal and Generic accounted for 58.9% of observations, so accuracy rewards correct majority predictions more heavily than improvements on rare attacks. Balanced accuracy measures the recovery of each class without a direct precision penalty. Together, the three measures separate accurate majority classification from broad attack recovery. Outer-fold scores were summarised as mean ± sample standard deviation. Per-class scores and confusion matrices used pooled outer-test predictions, because single test folds held too few Worms observations.

D. Tuning of the Earlier and Revised Models

Table III lists the earlier search domains. The earlier study evaluated 24 k-NN and 32 tree candidates per outer fold on 30 000 sampled training observations. Revised tuning used all outer-training observations and kept the earlier domains. Differences between baseline and revised models therefore reflected the training data and the search budget rather than new candidate values. The revised k-NN evaluated all 256 settings, whereas the tree evaluated 256 of 1 728 settings sampled without replacement with seed 21. The neighbour grid ranged from three to 21 neighbours, from neighbourhoods that follow individual training observations to neighbourhoods that average over many observations. Selection maximised the mean macro F-score over the three inner folds. The report calls this mean the inner macro F-score.

Table III
SEARCH DOMAINS OF THE NEAREST NEIGHBOUR AND TREE MODELS

Model	Parameter	Values
k-NN	Numeric scaling	standard, robust
	Rare-level pooling	none, 50
	Correlation pruning	none, 0.995
	Univariate selection	none, 30 features
	Neighbours k	3, 5, 11, 21
	Vote weighting	uniform, distance
	Minkowski order q	1, 2
Tree	Rare-level pooling	none, 50
	Impurity	Gini, entropy
	Maximum depth	none, 8, 16, 24
	Minimum split size	2, 10, 25
	Minimum leaf size	1, 5, 15
	Features for each split	all, square root of total
	Class weights	none, balanced
	Pruning penalty α	0, 10^{-4}, 10^{-3}

E. Questions and Ensemble Tuning

Nine questions, each with a prediction, were set before the first evaluation and guided the original tuning and per-class analysis. Table IV gives the comparisons and predictions. The analysis of Q9 reported absolute changes in the observed class F-scores.

For each outer fold, TabM stage one evaluated the 24 capacity settings of Q4 at $M = 32$, with shared batches and no class weights. Stage two changed one factor from the stage-one selection: $M \in \{1, 4, 8\}$, balanced weights or distinct batches. Each of the 29 candidates received three inner-fold fits. The highest inner macro F-score selected the final configuration. The grid contained the package defaults of width 512, depth three and dropout 0.1, with width and depth at the upper ends of their ranges. The reference size $M = 32$ followed Gorishniy *et al.*, who fixed M at 32 without tuning, and the learning-rate grid exceeded 10^{-3} because the same authors found higher optimal learning rates for TabM than for an MLP [6]. Weight decay, batch size, patience and the epoch limit stayed fixed, so the candidates differed only in the factors of Q1 to Q4.

The inner validation fold also selected the stopping epoch.

The meta-learner search crossed three representations, $C \in \{0.01, 0.1, 1, 10\}$ and absent or balanced class weights, giving 24 settings per stack. The C values formed a logarithmic grid around the scikit-learn default of one. Grouped three-fold cross-validation on the out-of-fold probability table scored each setting. This meta-validation split the table into meta-training and meta-validation partitions. The selected setting was refitted on that whole table.

F. Protocol Revision and Inner Selection

The first run was completed on 17 October 2026. The run scored TabM candidates on a grouped 20% hold-out and tuned the meta-learner on 30 000 observations. Candidate differences were comparable with hold-out variability. The final run used inner cross-validation, full-scale tuning of the earlier models and five-seed TabM averages trained without a final hold-out. The revised analysis also added a Holm adjustment of the

Table IV
TUNING QUESTIONS, COMPARISONS AND PREDICTIONS

	Question	Grid or comparison	Prediction
Q1	How much does the ensemble size improve the macro F-score over a single MLP?	$M \in \{1, 4, 8, 32\}$	Most of the gain arrives by $M = 8$.
Q2	Do balanced class weights raise the F-scores of Worms and Shellcode without hurting the majority classes?	no weights, balanced weights	Minority recall rises, but the precision on Normal and Generic falls.
Q3	Do members trained on distinct minibatches outperform members that share minibatches?	shared, distinct	Distinct and shared minibatches score alike, because the scaling vectors already make the members diverse.
Q4	What capacity suits data with label conflicts, and does stronger regularisation help?	width $\{256, 512\}$, depth $\{2, 3\}$, dropout $\{0, 0.1, 0.2\}$, learning rate $\{10^{-3}, 2 \times 10^{-3}\}$	A dropout rate above zero wins, and capacity matters little.
Q5	Does the learned meta-learner of S2 outperform the soft vote V2?	S2 against V2	S2 scores slightly higher, because averaging cannot down-weight the weak k-NN.
Q6	Does TabM add useful diversity to the stack?	S2 against S1, error correlation	TabM adds diversity, because TabM errors correlate less with tree errors than k-NN errors do.
Q7	Are the own-class coefficients of k-NN largest on DoS and Analysis, where k-NN previously outperformed the tree?	own-class coefficients	The k-NN coefficients are largest for DoS and Analysis.
Q8	What meta-feature representation works best?	probabilities, log-probabilities, one-hot labels	Probabilities or log-probabilities score highest, because one-hot labels discard confidence.
Q9	How do the models change the F-scores of Backdoor, DoS and Analysis relative to the earlier tree?	per-class F-scores	Gains are limited on these classes and occur mainly on the other attack classes.

corrected-test p-values across the 11 comparisons. The grids and original predictions were retained, but the first outer-test results were already known.

The meta-validation did not rebuild the base probability table inside each meta-training partition. Base fits producing meta-training features had therefore used labels from the meta-validation partition. Base configurations and epoch counts were also selected using all inner folds. The resulting inner macro F-scores were selection criteria rather than independent estimates of the complete stack. This dependence did not introduce outer-test observations into the final-run fits.

G. Statistical Comparisons

Each comparison used five paired outer-fold differences in macro F-score. Each of the other seven models was compared with the baseline tree. The revised k-NN was compared with the baseline k-NN. Stack S2 was compared with S1, V2 and TabM. Exact two-sided Wilcoxon tests were computed, but five folds cannot give $p < 0.0625$, so Table VI reports the number of folds above zero instead. Corrected resampled t-tests accounted approximately for overlapping training sets. The Holm procedure adjusted the corrected-test p-values across all 11 comparisons. Both raw and adjusted values were retained, with adjusted $p < 0.05$ required for a family-level significance claim.

Grouped folds preserved direct train–test separation and paired the earlier and revised models.

V. RESULTS AND DISCUSSION

This section compares the selected models, explains the class-specific trade-offs and answers the tuning questions. The interpretation links each learning rule to the observed scores, then considers computational cost and the limits of the evidence.

A. Selected Configurations

Table V gives the settings that differed between outer folds. Every revised tree used balanced class weights and a minimum leaf size of one, whereas the baseline tree used balanced weights on fold five only. Re-tuning therefore introduced class weighting on four of the five folds.

Every revised k-NN retained robust scaling, Manhattan distance and distance weighting. No revised k-NN used correlation pruning or univariate selection. Manhattan distance sums absolute differences without squaring the differences, so one feature with large scaled values dominates the distance less than under the Euclidean distance, a plausible reason for the selection. The tested feature-removal rules therefore did not improve the selected full-scale configurations. The baseline k-NN used $k = 3$ on four folds, whereas every revised k-NN selected $k = 5$ or $k = 11$. Each revised inner fit held about 137 000 observations, against 20 000 in the earlier study. A neighbourhood with the same k therefore covered a smaller region of the feature space. The larger selected neighbourhoods were consistent with averaging over noisy labels without losing locality.

Every selected TabM had depth three. Within each outer fold, the two best candidates differed by at most 0.003 in inner macro F-score, whereas the standard deviation of one candidate over the inner folds ranged from 0.005 to 0.013. The configuration changes across outer folds were therefore within tuning variability. Both stacks selected probabilities and no class weights on every fold.

Table V
CONFIGURATIONS SELECTED FOR EACH OUTER FOLD

Setting	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Revised tree					
Impurity	Gini	entropy	entropy	entropy	entropy
Maximum depth	24	16	16	16	16
Minimum split size	2	10	10	10	10
Pruning penalty α	0	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Rare-level pooling	none	50	50	50	50
Revised k-NN					
Neighbours k	11	11	5	5	11
Rare-level pooling	none	none	50	50	50
TabM					
Members M	32	8	32	32	8
Width	256	512	256	512	512
Dropout rate	0.1	0	0	0	0
Learning rate	0.002	0.002	0.002	0.001	0.002
Meta-learner of stack S2					
Inverse regularisation C	0.1	0.1	1	1	0.01

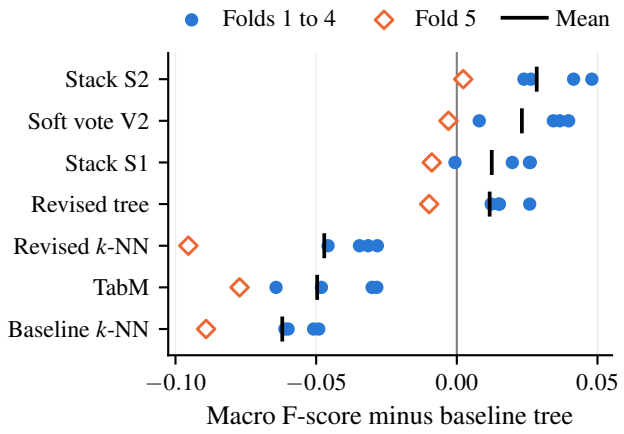


Figure 1. Outer-fold differences in macro F-score from the baseline tree

B. Overall Performance

Stack S2 had the highest mean macro F-score and accuracy in Table VI. Figure 1 shows that S2 scored above the baseline tree on every fold. The exact Wilcoxon test reached the five-fold minimum of $p = 0.0625$, as did five other comparisons with differences of one sign on every fold. The corrected test did not establish the difference, before or after the Holm adjustment. Only two of the 11 comparisons remained significant after adjustment. The baseline k-NN scored below the baseline tree, and S2 scored above TabM.

The three measures ranked the models differently. The revised tree had the highest balanced accuracy but the lowest accuracy. Recovering more minority-class observations therefore came with more errors on frequent classes. TabM had the highest accuracy of the base learners but a lower macro F-score than either tree. Accurate classification of frequent classes did not compensate for weak rare-class recovery under macro averaging.

Re-tuning raised the mean macro F-score of both baseline models, with gains on four folds and no significant difference. Fold five gave the smallest S2 gain over the baseline tree. The baseline tree of fold five already used balanced weights, so the fold compared S2 with a stronger minority-sensitive baseline. Other tree settings also differed on fold five, so the comparison did not isolate the effect of weighting.

Stack S1 scored within 0.001 of the revised tree, so the revised k-NN added almost nothing to the revised tree. Adding TabM raised S2 above S1 on every fold. The raw corrected test was significant, but the Holm-adjusted test was not. The improvement therefore supported further study of TabM as a complementary learner, without a family-level significance claim.

C. Inductive Bias and Class-Specific Performance

Table VII shows why the aggregate rankings differed. The baseline tree exceeded the baseline k-NN on eight classes, with the largest advantages on Worms and Shellcode. A tree selects different features in different branches and forms conjunctions of threshold and category tests. A k-NN classifier instead applies one distance rule across all encoded features [1], [8]. Uninformative coordinates and overlapping classes therefore enter every neighbourhood, even where a small feature subset identifies a useful rule. Table VIII gives the precision and recall behind the F-scores of four classes that show the main precision–recall trade-offs.

Feature-specific partitioning was a plausible explanation for the stronger rare-class results. The baseline k-NN recovered fewer than half as many Worms observations as the baseline tree. Worms supplied few neighbours to any query observation, whereas nearby observations of frequent classes competed in every vote. The revised k-NN selected larger neighbourhoods on four of the five folds and recovered still fewer Worms observations, as the bias of a large k predicts. The results supported the tested tree more strongly than the tested distance model for Worms and Shellcode detection. Feature ablations would be needed to identify the responsible predictors.

The tree advantage was not uniform. The baseline k-NN scored higher on DoS and Analysis, so local similarity retained class information that the selected tree did not exploit equally well. Revising the tree more than doubled DoS recall at slightly lower precision. Balanced weights increased the influence of minority observations during both split selection and leaf prediction. The recall gain matched that mechanism, although other revised settings changed too.

Higher recall did not guarantee a higher F-score. On Worms, the revised tree gained recall but lost precision. The Worms F-score of the revised tree fell below the Worms F-score of the baseline tree. On Normal, recall fell whereas precision rose. The revised tree therefore issued more minority predictions and required stronger evidence before predicting Normal. These trade-offs explained the rise in balanced accuracy and the fall in accuracy. The precision losses also explained why the revised tree did not improve every minority F-score.

TabM showed the opposite imbalance. On DoS and Worms, TabM had the highest precision of all models but recovered fewer than one in six observations of either class. The net-

Table VI
OUTER-FOLD PERFORMANCE AND PAIRED COMPARISONS OF THE MODELS

Model	Macro F-score	Balanced accuracy	Accuracy	Difference	Folds above zero	Corrected p	Holm p
Models, with paired differences from the baseline tree							
Baseline tree	0.575 ± 0.017	0.583 ± 0.051	0.811 ± 0.023	reference	reference	reference	reference
Baseline k-NN	0.513 ± 0.010	0.518 ± 0.008	0.785 ± 0.012	-0.062 ± 0.016	0	0.005	0.045
Revised tree	0.586 ± 0.009	$\mathbf{0.680} \pm 0.024$	0.761 ± 0.007	$+0.012 \pm 0.013$	4	0.255	0.766
Revised k-NN	0.528 ± 0.014	0.510 ± 0.013	0.802 ± 0.003	-0.047 ± 0.028	0	0.065	0.454
TabM	0.525 ± 0.009	0.509 ± 0.006	0.825 ± 0.002	-0.050 ± 0.021	0	0.025	0.228
Stack S1	0.587 ± 0.007	0.573 ± 0.011	0.824 ± 0.002	$+0.012 \pm 0.016$	3	0.316	0.766
Stack S2	$\mathbf{0.603} \pm 0.010$	0.587 ± 0.011	$\mathbf{0.829} \pm 0.002$	$+0.028 \pm 0.018$	5	0.076	0.455
Soft vote V2	0.598 ± 0.007	0.601 ± 0.007	0.819 ± 0.002	$+0.023 \pm 0.019$	4	0.149	0.744
Other paired comparisons							
Revised k-NN − baseline k-NN				$+0.015 \pm 0.014$	4	0.178	0.744
Stack S2 − stack S1				$+0.016 \pm 0.007$	5	0.032	0.257
Stack S2 − soft vote V2				$+0.005 \pm 0.009$	4	0.413	0.766
Stack S2 − TabM				$+0.078 \pm 0.013$	5	0.001	0.010

Table VII
POOLED OUTER-TEST F-SCORE OF EACH CLASS

Class	Baseline tree	Baseline k-NN	Revised tree	Revised k-NN	TabM	Stack S1	Stack S2	Soft vote V2
Normal	0.909	0.900	0.856	0.904	0.918	0.917	**0.922**	0.916
Reconnaissance	0.831	0.713	0.825	0.757	0.832	0.835	**0.840**	0.832
Backdoor	0.152	0.095	0.160	0.106	0.160	0.166	0.178	**0.233**
DoS	0.311	0.403	**0.472**	0.419	0.102	0.303	0.309	0.433
Exploits	0.716	0.683	0.645	0.697	**0.738**	0.729	0.738	0.719
Analysis	0.104	**0.155**	0.155	0.109	0.061	0.109	0.104	0.138
Fuzzers	0.606	0.552	0.616	0.563	0.597	0.617	0.631	**0.650**
Worms	0.598	0.288	0.561	0.275	0.260	0.580	**0.661**	0.440
Shellcode	0.615	0.349	0.582	0.476	0.602	0.633	**0.660**	0.631
Generic	0.987	0.985	0.987	0.986	0.988	0.988	**0.989**	0.988
Macro average	0.583	0.512	0.586	0.529	0.526	0.588	**0.603**	0.598

Table VIII
POOLED OUTER-TEST PRECISION AND RECALL OF FOUR CLASSES

	Normal		Backdoor		DoS		Worms	
Model	Precision	Recall	Precision	Recall	Precision	Recall	Precision	Recall
Baseline tree	0.910	0.908	0.143	0.163	0.392	0.258	0.598	0.598
Baseline k-NN	0.885	0.914	0.093	0.097	0.370	0.443	0.385	0.230
Revised tree	0.975	0.764	0.097	0.454	0.363	0.674	0.440	0.776
Revised k-NN	0.885	0.924	0.319	0.064	0.376	0.472	0.542	0.184
TabM	0.895	0.943	0.739	0.090	0.676	0.055	0.794	0.155
Stack S1	0.907	0.927	0.695	0.094	0.418	0.238	0.643	0.529
Stack S2	0.910	0.934	0.775	0.100	0.428	0.242	0.673	0.649
Soft vote V2	0.928	0.904	0.211	0.260	0.384	0.497	0.628	0.339

work therefore made selective predictions for these attacks and missed most of the observations. The unweighted cross-entropy gave frequent labels more influence over the total training objective. Shared neural weights coupled decisions across the feature space, whereas tree predictions used separate leaf proportions. These mechanisms were consistent with strong majority classification and weak minority recovery.

Grinsztajn *et al.* linked neural-network difficulties on tabular data to irregular target functions and uninformative features [7]. The tree–TabM comparison was consistent with that account, but did not test smoothness or feature relevance directly. TabM also scored well on Exploits and Reconnaissance, so the result did not show a general failure to learn the supplied traffic attributes.

Figure 2 compares the baseline tree and S2. Stack S2 recognised more Exploits observations than the baseline tree. Stack S2 also assigned more DoS and Backdoor observations to Exploits. The stack therefore did not resolve the main confusion among these classes. Exploits observations outnumbered DoS observations by 2.7 to one. Within the conflicting groups that contained DoS observations, 64% of the non-DoS observations belonged to Exploits. A plausible mechanism is that a meta-learner fitted by likelihood assigns observations that resemble both classes to the more frequent label. The

largest S2 F-score gains over the tree were on Worms and Shellcode, the two smallest classes. Macro averaging gave those gains the same influence as gains of equal size on Normal.

For Q9, the prediction largely held for S2, whose largest gains over the baseline tree were on Worms and Shellcode and whose DoS and Analysis scores barely changed. The prediction failed for the revised tree and V2, whose largest gains were on DoS. The soft vote had the highest Backdoor F-score of all models. The revised tree alone remained best on DoS. On Backdoor, the revised tree recovered many observations with low precision, whereas TabM recovered few observations with high precision. Averaging these assignments with the k-NN probabilities gave V2 a higher Backdoor F-score than any member. The Backdoor gain therefore came from the combination. The study did not measure the contribution of class weighting alone.

Analysis remained difficult for every model, and no model reached an F-score of 0.16. Most Analysis observations lay in conflicting groups, so class identity was frequently ambiguous from the supplied inputs. The experiment did not separate that ambiguity from model or tuning limitations. The class-specific results contradicted a uniform ensemble advantage.

D. Neural Ensemble Tuning

For Q1, the ensemble score rose with the number of members in Figure 3, whereas the mean score of a single member stayed flat. Combining outputs therefore improved the prediction without improving the average member, consistent with averaging partly different errors [4]. About 60% of the observed gain arrived by eight members, in line with the Q1 prediction.

The smaller standard deviation at 32 members described variation across outer-fold inner macro F-scores and did not establish a reduced training-seed variance. The 32-member point was also selected from 24 capacity candidates, whereas the smaller ensembles reused the selected capacity. Capacity selection therefore made the estimated size effect optimistic.

Balanced weights lowered the inner macro F-score of TabM, as Table IX shows. The Worms and Shellcode F-scores fell most, so the answer to Q2 was negative. Tuning recorded class F-scores only, so the predicted recall and precision changes could not be checked. Equalising the total class weight did not equalise uncertainty. An inner fit contained only about 93 Worms observations. Each Worms observation received roughly 148 times the unit loss weight, against 0.28 for each Normal observation. Repeated gradients from a small set of rare observations consequently had much greater influence.

A plausible explanation was that weighting increased rare-class predictions beyond the precision–recall balance favoured by macro F-score. Tuning did not record class-wise precision, so that mechanism remained unverified for TabM. The tree results demonstrated the relevant trade-off directly, with higher rare-class recall accompanied by lower precision. Weighting acts differently on neural gradients and on tree partitions, a possible reason that the weighting choice did not transfer between model families.

Table IX
INNER F-SCORES OF TABM WITHOUT AND WITH BALANCED CLASS WEIGHTS

Class	No weights	Balanced weights	Change
Normal	0.914	0.847	−0.067
Reconnaissance	0.827	0.818	−0.008
Backdoor	0.140	0.162	+0.022
DoS	0.361	0.468	+0.107
Exploits	0.716	0.596	−0.120
Analysis	0.067	0.170	+0.103
Fuzzers	0.578	0.604	+0.026
Worms	0.257	0.093	−0.164
Shellcode	0.576	0.413	−0.163
Generic	0.986	0.986	0.000
Macro average	0.542	0.516	−0.027

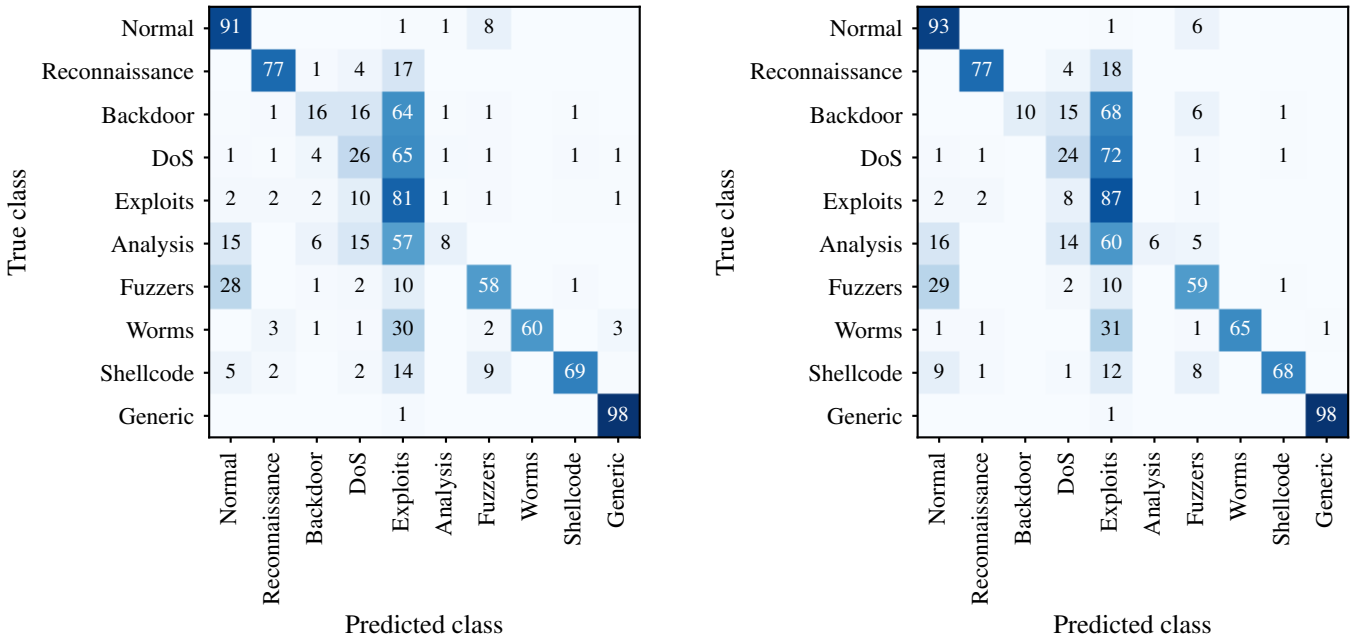
Distinct minibatches lowered the inner macro F-score on every fold, by 0.009 on average. The mean inner macro F-score of a single member fell from 0.535 to 0.528. The Q3 comparison therefore did not support an advantage from separate shuffles. Gorishniy *et al.* instead reported that shared batches caused only a minor average loss [6], whereas distinct batches lowered the score here on every fold. Every member still saw every observation each epoch, so this experiment was not bootstrap bagging. Separate shuffles changed the observations contributing to a shared-weight update as well as the similarity between members. The weaker members indicated that additional batch diversity did not offset the changed optimisation.

For Q4, wider and deeper networks scored better on average in Table X, consistent with additional capacity for interactions among traffic attributes. A dropout rate of zero scored highest. Stronger dropout delayed the best epoch. The dropout part of the Q4 prediction therefore failed, and the capacity part held only partly, because width and depth changed the mean inner macro F-score less than dropout did. Early stopping and weight sharing already constrained fitting. Extra dropout was a plausible source of unnecessary restriction [5], [6]. The experiment did not isolate the regularisation mechanisms.

Capacity effects were small relative to the gap between TabM and the best stack. No tested factor changed the mean

Table X
INNER MACRO F-SCORE AND BEST EPOCH OF TABM BY CAPACITY LEVEL

Factor	Level	Inner macro F-score	Best epoch
Width	256	0.524	21.7
	512	0.531	17.2
Depth	2	0.526	19.0
	3	0.530	19.8
Dropout	0	0.533	16.9
	0.1	0.527	18.0
	0.2	0.522	23.4
Learning rate	10^{-3}	0.526	21.4
	2×10^{-3}	0.530	17.4



(a) Baseline tree

(b) Stack S2

Figure 2. Row-normalised confusion matrices of the pooled outer-test predictions in percent

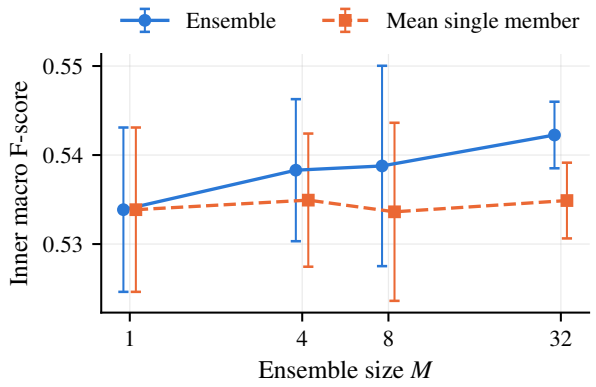


Figure 3. Inner macro F-score against ensemble size, with standard deviations over five outer folds

inner macro F-score by more than 0.012. Increasing capacity within the grid therefore did not remove the model-selection and minority-class trade-offs. The result applied only to the tested widths and depths.

The first-run TabM had reached a macro F-score of 0.549 and a DoS F-score of 0.409, both above the final values. The final run trained without validation checkpoint selection. A diagnostic on fold one found low DoS recall in single-seed TabMs, so seed averaging did not cause the low recall. Removal of checkpoint selection remained a plausible explanation, because validation macro F-score rewards recovery of small classes even when the training likelihood favours frequent labels. The tuning, configurations and training partitions also changed, so the study did not attribute the low recall to the removal of checkpoint selection alone.

E. Stacking and Complementary Predictions

Stack S2 exceeded soft vote V2 by the smallest mean difference of the 11 paired comparisons in Table VI. The Q5 comparison therefore did not establish an advantage from learning the combination. Equal-weight averaging already combined the revised tree with the more selective TabM predictions. Logistic regression added class-specific and cross-class corrections, but the extra parameters produced little mean improvement over averaging.

The two combinations favoured different outcomes. Stack S2 had the higher accuracy, whereas soft vote V2 had the higher balanced accuracy. Stack S2 consequently made fewer total mistakes but had a lower mean class recall. Per class, S2 scored far higher than V2 on Worms but lower on Backdoor, DoS, Analysis and Fuzzers. Over the other nine classes, the mean pooled F-score of V2 exceeded the mean pooled F-score of S2 by 0.019. The S2 advantage rested on Worms, the smallest class. The fitted combination changed the balance separately for each class, whereas V2 retained the minority-class probabilities of the revised tree through a fixed equal-weight average.

For Q6, TabM and the revised tree had the lowest error correlation and the lowest double fault in Table XI. TabM therefore frequently supplied a correct prediction where the revised tree failed, despite a lower standalone macro F-score.

Lower error correlation was a useful diagnostic of complementarity, but did not isolate the cause of the stack gain [10].

The high recall of the revised tree and the high precision of TabM were a plausible source of the S2 gain over S1. The first-run error correlation of 0.605 paired TabM with the baseline tree, and the same pair gave 0.676 in the final run. The low correlation in Table XI therefore came mainly from the revised tree rather than from changes to TabM. Diversity thus depended on the evaluated configurations rather than on model names alone.

For Q7, TabM had the largest standardised own-class coefficient on eight classes in Figure 4, with the tree largest on Worms and Shellcode. The k-NN value exceeded the tree value on DoS, but not on Analysis. The Q7 prediction therefore did not hold, because TabM rather than k-NN had the largest coefficients on DoS and Analysis.

These coefficients described one contribution to a class score. All base probabilities and competing class scores entered the final decision, and the probabilities from each base learner summed to one. A large coefficient did not establish that a learner dominated predictions. TabM supplied graded DoS probabilities even when DoS was not the largest output. Stacking retained probability variation that predicted labels alone would discard.

For Q8, probabilities scored highest for both stacks in Table XII. The table averages the best inner macro F-score of each representation over the outer folds. The result supported the value of confidence information, in agreement with Ting and Witten [17]. Of the revised k-NN probabilities, 82% were zero. The revised k-NN assigned probability one to a single class for 58% of observations. The logarithm mapped zeros to about -9.2, magnifying differences among these sparse estimates. That transformation did not improve the tested linear meta-learner, although probability calibration was not measured.

F. Computational Cost

Table XIII separates tuning from the three inner refits and the outer-training refit. Each TabM refit included five seeds, so four refits represented 20 trained TabMs. The revised k-NN and TabM searches shared a GPU, so the recorded durations should not be added to estimate elapsed runtime. Each of the two searches alone took more than 200 times as long per outer fold as the four baseline tree fits.

Stack S2 therefore exchanged more than 200 times the fitting and selection cost of the baseline tree for a gain that was not significant. A single tree remained attractive when retraining cost mattered. Stack S2 would be justified by the measured class-specific gains only if the application valued those gains enough to support the extra computation. Inference latency was not measured.

G. Limitations

The protocol changed after the first outer-test results were known. The final scores therefore described an exploratory comparison on reused test folds, even though outer-test observations remained outside final-run fitting. Five dependent fold differences also gave low power. Neither S2 against the baseline tree nor S2 against S1 was significant after adjustment across the 11 comparisons. Repeated grouped evaluation under a fixed protocol would test whether the gains persisted.

Inner macro F-scores included optimism from checkpoint selection and dependence within meta-validation. The optimism was not necessarily equal across candidates. These

Table XI
DIVERSITY OF BASE-LEARNER PAIRS ON THE OUTER-TEST FOLDS

Pair	Error correlation	Label disagreement (%)	Double fault (%)
k-NN and tree	0.460	23.8	12.5
k-NN and TabM	0.565	16.3	12.0
Tree and TabM	0.307	28.5	9.2

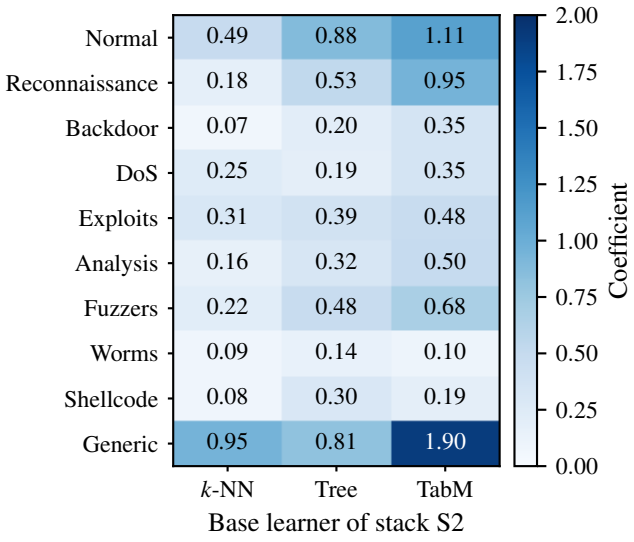


Figure 4. Own-class coefficients of the stack meta-learner averaged over five outer folds

Table XII
BEST INNER MACRO F-SCORE OF EACH META-FEATURE REPRESENTATION

Representation	Stack S1	Stack S2
Probabilities	0.578 ± 0.006	0.590 ± 0.005
Log-probabilities	0.561 ± 0.005	0.577 ± 0.004
One-hot labels	0.567 ± 0.010	0.559 ± 0.009

Table XIII
COMPUTING TIME FOR EACH OUTER FOLD IN SECONDS

Model	Device	Tuning	Four final fits
Baseline tree	CPU	–	19
Baseline k-NN	CPU	–	676
Revised tree	CPU	185	16
Revised k-NN	GPU	4 617	216
TabM	GPU	4 977	405
Meta-learner of S2	CPU	64	–

limitations constrained causal interpretation of tuning effects. Independent checkpoint validation and rebuilding base predictions within meta-training partitions would provide cleaner comparisons.

The study covered one traffic dataset, a finite neural-capacity grid and a linear meta-learner. Robust scaling also left `response_body_len` unscaled, so response size supplied most of the numeric part of the selected k-NN distances between observations with a response body. A logarithmic transformation before scaling was not tested. Differences in weighting, tuning budgets and final training prevented attribution of score gaps to inductive bias alone. The proposed mechanisms were supported by observed error patterns, but required controlled ablations to separate model structure from training choices.

The highest-scoring stack combined learners with different precision–recall trade-offs. The S2 gains over the baseline tree concentrated on Worms and Shellcode, whereas the soft vote kept the highest Backdoor and Fuzzers F-scores and the revised tree remained best on DoS and balanced accuracy.

VI. Conclusions

The study compared neural network ensembles and stacking with k-nearest neighbours (k-NN) and a decision tree on network traffic classification. TabM combined multilayer perceptrons (MLPs) through shared weights. Stacking combined the probabilities of TabM and the re-tuned earlier models. Nine questions guided tuning and class-specific analysis on the earlier five grouped outer folds.

The three-learner stack had the highest mean macro F-score and exceeded the baseline tree on every fold, but the gain was not statistically significant. The baseline tree recovered Worms and Shellcode more effectively than the baseline k-NN under the tested settings. Feature-specific partitioning provided a plausible explanation for that advantage over a single distance rule. The revised, class-weighted tree raised minority recall but also issued more false attack predictions. TabM was more selective on rare classes and more accurate on frequent classes.

TabM alone scored below both trees but improved the stack on every fold. The low error correlation with the revised tree supported the usefulness of complementary predictions. Learned stacking did not establish an advantage over equal-weight averaging, and the small lead of the stack over the soft vote rested on Worms alone. The meta-learner gave TabM the largest own-class coefficient on most classes, contrary to the predicted k-NN emphasis on DoS and Analysis. Probabilities were the most effective meta-features. The soft vote improved Backdoor F-score beyond every base learner, whereas the revised tree remained best on denial of service (DoS). No ensemble improved every class.

More TabM members gave a small gain in inner macro F-score. Balanced weights, distinct minibatches and dropout lowered the inner macro F-score of TabM. The fall in final DoS recall was already present before seed averaging, but the experiment did not isolate the responsible training change. These findings applied to the selected configurations and the protocol revised after the first test results.

Future work should repeat the grouped evaluation under a fixed protocol and run matched ablations of weighting and checkpoint selection. Class-specific thresholds selected on independent inner validation would test the minority-recall trade-off directly. Scoring the class-weighted meta-learners of the existing grid by an explicit recall or cost criterion would separate the application objective from the unweighted likelihood.

Appendix A
Acronyms

Acronym	Meaning
Adam	adaptive moment estimation
AdamW	Adam with decoupled weight decay
CPU	central processing unit
DoS	denial of service
GPU	graphics processing unit
k-NN	k-nearest neighbours
L-BFGS	limited-memory Broyden–Fletcher–Goldfarb–Shanno
MLP	multilayer perceptron
ReLU	rectified linear unit
UNSW	University of New South Wales

APPENDIX B
SYMBOLS

Symbol	Meaning
$\mathbf{a}$, $\mathbf{a}'$	input and output of a hidden layer
$\mathbf{a}_h$	input of member h to a BatchEnsemble layer
$\mathbf{b}$	bias vector of a hidden layer
$\mathbf{b}_h$	bias vector of member h
C	inverse regularisation strength
D_v	difference in macro F-score on fold v
$\bar{D}$	mean of the fold differences
d	distance function
e	leaf impurity of a tree, weighted by leaf share
F_l	F-score of class l
$\bar{F}$	macro F-score
G	Gini impurity
H	entropy
h	member index
i	observation index
j	feature index
K	number of classes
k	number of neighbours
l, l'	class indices
$\mathcal{L}$	TabM loss of one observation
M	number of TabM members
m	number of features
N_k	index set of the k nearest neighbours
n	number of training observations
n_l	number of training observations of class l
n_t	training weight in leaf t
$\mathbf{o}_h$	output of member h of a BatchEnsemble layer
P_l	precision of class l
p	probability value of a test
Q	impurity of a node
ΔQ	impurity decrease of a test
q	order of the Minkowski distance
R_l	recall of class l
$\mathbf{r}_h$, $\mathbf{s}_h$	input and output scaling vectors of member h
S_t	training observations at node t
S_L, S_R	training observations of the left and right child
T	subtree of a pruned tree
$\lvert T \rvert$	number of leaves of T
t	tree node
t_L, t_R	left and right child nodes
V	number of cross-validation folds
v	fold index
$\mathbf{W}$	weight matrix of a layer
$\mathbf{W}_h$	effective weight matrix of member h
w_i	vote weight of neighbour i
$\mathbf{x}$, $\mathbf{x}'$	predictor vectors
x_j	feature j of a predictor vector
y, $\hat{y}$	true and predicted class
$\mathbf{z}$	meta-feature vector
α	pruning penalty for each leaf
$\boldsymbol{\beta}_l$, $\beta_{l,0}$	meta-learner coefficients and intercept
δ	Kronecker delta
θ	split threshold
$\pi_l(t)$	proportion of class l at node t
$\hat{\pi}_l$	estimated probability of class l
$\hat{\pi}_{h,l}$	probability of class l from member h
$^\intercal$	transpose
$\hat{\sigma}^2$	sample variance of the fold differences
τ	corrected resampled t statistic
ω_l	weight of class l
$\odot$	elementwise product

REFERENCES

[1] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[2] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.

[3] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[4] T. G. Dietterich, "Ensemble methods in machine learning," in *Proceedings of the First International Workshop on Multiple Classifier Systems*, ser. Lecture Notes in Computer Science, vol. 1857, Cagliari, Italy, 2000, pp. 1–15.

[5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[6] Y. Gorishniy, A. Kotelnikov, and A. Babenko, "TabM: Advancing tabular deep learning with parameter-efficient ensembling," in *Proceedings of the 13th International Conference on Learning Representations*, Singapore, 2025.

[7] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proceedings of the 36th Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*, ser. Advances in Neural Information Processing Systems, vol. 35, New Orleans, LA, 2022, pp. 507–520.

[8] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.

[9] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[10] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," *Machine Learning*, vol. 51, no. 2, pp. 181–207, 2003.

[11] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proceedings of the 7th International Conference on Learning Representations*, New Orleans, LA, 2019.

[12] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.

[13] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proceedings of the 2015 Military Communications and Information Systems Conference*, Canberra, Australia, 2015, pp. 1–6.

[14] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[15] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Proceedings of the 33rd Conference on Neural Information Processing Systems*, ser. Advances in Neural Information Processing Systems, vol. 32, Vancouver, Canada, 2019, pp. 8024–8035.

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] K. M. Ting and I. H. Witten, "Issues in stacked generalization," *Journal of Artificial Intelligence Research*, vol. 10, pp. 271–289, 1999.

[18] Y. Wen, D. Tran, and J. Ba, "BatchEnsemble: An alternative approach to efficient ensemble and lifelong learning," in *Proceedings of the 8th International Conference on Learning Representations*, Addis Ababa, Ethiopia, 2020.

[19] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[20] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.